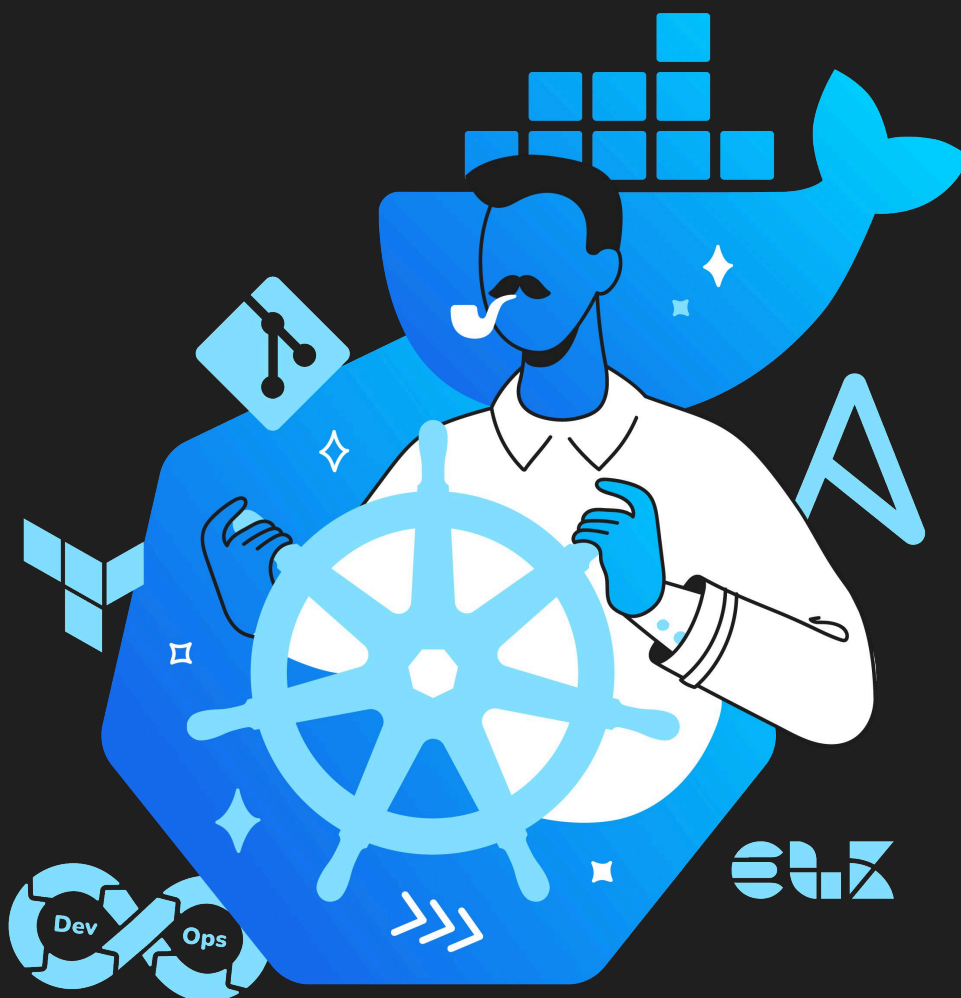


Методический материал

РАБОТА С DOCKER SWARM

Урок 8.





До начала работ, убедитесь что у вас установлен Docker. Docker swarm является встроенным в дистрибутив Docker решением по оркестрации, поэтому предпринимать дополнительных действий по его установке не потребуется.

Для помощи в установке Docker можно воспользоваться методическими материалами предыдущих уроков.



Как только Docker будет установлен, переходите к следующему разделу по работе с Docker swarm.

Запускаем кластер Docker swarm:

```
$ docker swarm init
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker swarm init
Swarm initialized: current node (vnzq9rhegg5zc8ljnkd2fd3e) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1mmzh10ap1j97hlm5d6t0z0j7fet9du7irwak6qozusoqpjqey-
9uho8xg028c6rwhb8gv1nlfb4 192.168.1.11:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```



Кластер Docker swarm успешно проинициализирован.

Выполним команду просмотра статуса узлов запущенного кластера:

```
$ docker node ls
```

```
flllpp@user-IdeaPad-3-15IAU7:~$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STA
vnzq9rhegg5zc8ljnkd2fd3e *	user-IdeaPad-3-15IAU7	Ready	Active	Leader

28.0.4

Убеждаемся в работоспособности узлов кластера Docker swarm.

Запускаем деплой сервиса Nginx в Docker swarm:

```
$ docker service create --name nginx -p 8080:80 nginx:alpine
```

```
flllpp@user-IdeaPad-3-15IAU7:~$ docker service create --name nginx -p 8080:80 nginx:alpine
wo99bund3u3r4038y4w5hgxp9
overall progress: 1 out of 1 tasks
1/1: running
verify: Service wo99bund3u3r4038y4w5hgxp9 converged
```



Сервис успешно запущен.

Выполним команду просмотра запущенных Docker-контейнеров:

```
$ docker ps
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
76c706d4f2ad   nginx:alpine   "/docker-entrypoint..." 32 seconds ago Up 31 seconds 80
/tcp          nginx.1.mmqx2qvtg8bprq@ihqsqapbw
```

Docker-контейнер успешно запущен.

Проверяем доступность сервиса, отправляя команду Curl:

```
$ curl 127.0.0.1:8080
```

```
filipp@user-IdeaPad-3-15IAU7:~$ curl 127.0.0.1:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```



Убеждаемся в работоспособности сервиса.

Запускаем деплой еще одного сервиса (Redis):

```
$ docker service create --name redis redis:6
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service create --name redis redis:6
zgir14ih2ztzyumifqr28czjj
overall progress: 1 out of 1 tasks
1/1: running
verify: Service zgir14ih2ztzyumifqr28czjj converged
```

Сервис успешно запущен.

Выполним команду просмотра запущенных Docker-контейнеров:

```
$ docker ps
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
c19fcd487289	redis:6	"docker-entrypoint.s..."	About a minute ago	Up About a minute
6379/tcp	redis.1.sly5xlzsuc1w9erynvm4htm45			
76c706d4f2ad	nginx:alpine	"/docker-entrypoint...."	51 minutes ago	Up 51 minutes
80/tcp	nginx.1.mmqx2qvtg8bprq0ihqsqapbw			



Оба сервиса (Nginx и Redis) успешно запущены.

Выполним команду для просмотра списка запущенных в Docker-swarm сервисов:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
ID                NAME      MODE          REPLICAS  IMAGE          PORTS
wo99bund3u3r     nginx    replicated    1/1        nginx:alpine   *:8080->80/tcp
zg1r14ih2ztz     redis    replicated    1/1        redis:6
```



Сервисы (Nginx и Redis) в Docker-swarm успешно запущены.

Проинспектируем сервис Redis в Docker swarm в Pretty-формате вывода:

```
$ docker service inspect --pretty redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service inspect --pretty redis

ID:                zgir14ih2ztzyumifqr28czjj
Name:              redis
Service Mode:     Replicated
  Replicas:        1
Placement:
UpdateConfig:
  Parallelism:     1
  On failure:      pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Update order:    stop-first
RollbackConfig:
  Parallelism:     1
  On failure:      pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Rollback order:  stop-first
ContainerSpec:
  Image:           redis:6@sha256:3a2bd6da7a0ad2a1786539c1d340ff9bd8ba6c321
09
  Init:            false
Resources:
Endpoint Mode:    vip
```



Обратим внимание на количество репликаций сервиса.

Выполним команду скалирования репликаций сервиса Redis:

```
$ docker service scale redis=2
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service scale redis=2
redis scaled to 2
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service redis converged
```

Репликации сервиса успешно скалированы до двух.

Выполним команду просмотра запущенных Docker-контейнеров:

```
$ docker ps
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
c3d65f0700d4	redis:6	"docker-entrypoint.s..."	24 seconds ago	Up 23 seconds	6379/
tcp	redis.2.ls8xu6tnz7urecmlcne1v5oec				
c19fcd487289	redis:6	"docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	6379/
tcp	redis.1.sly5xlzsuc1w9erynvm4htm45				
76c706d4f2ad	nginx:alpine	"/docker-entrypoint...."	53 minutes ago	Up 53 minutes	80/tc
p	nginx.1.mmqx2qvtvg8bprq0ihqsqapbw				



Обращаем внимание, что контейнеров Redis теперь два.

Повторим команду для просмотра списка запущенных в Docker-swarm сервисов:

```
$ docker service ls
```

```
flllpp@user-IdeaPad-3-15IAU7:~$ docker service ls
ID                NAME      MODE     REPLICAS  IMAGE      PORTS
wo99bund3u3r     nginx    replicated 1/1        nginx:alpine *:8080->80/tcp
zg1r14ih2ztz     redis    replicated 2/2        redis:6
```

В выводе отображена информация об изменившемся количестве репликаций сервиса Redis.

Обновим сервис Redis на новую версию используемого образа:

```
$ docker service update --image redis:7 redis
```

```
flllpp@user-IdeaPad-3-15IAU7:~$ docker service update --image redis:7 redis
redis
overall progress: 1 out of 2 tasks
1/2: running
2/2: ready
```



Обращаем внимание на минимальную задержку между последовательным обновлением двух репликаций сервиса.

Выполним команду для просмотра списка запущенных в Docker-swarm сервисов, чтобы убедиться в успешности обновления версии Redis:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
ID                NAME      MODE      REPLICAS  IMAGE          PORTS
wo99bund3u3r     nginx    replicated 1/1        nginx:alpine   *:8080->80/tcp
zg1r14h2ztz      redis    replicated 2/2        redis:7
```

Версия Redis успешно обновлена до V7.

Модифицируем сервис для введения задержки при обновлении реплик Redis при совершении Rolling-update:

```
$ docker service update --update-delay 10s redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service update --update-delay 10s redis
redis
overall progress: 2 out of 2 tasks
1/2: running
2/2: running
verify: Waiting 1 seconds to verify that tasks are stable...
```



Команда успешно применена.

Проинспектируем сервис Redis в Docker swarm в Pretty-формате вывода:

```
$ docker service inspect --pretty redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service inspect --pretty redis
ID:                zgir14ih2ztzyumifqr28czjj
Name:              redis
Service Mode:      Replicated
  Replicas:        2
Placement:
UpdateConfig:
  Parallelism:      1
  Delay:            10s
  On failure:       pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Update order:     stop-first
RollbackConfig:
  Parallelism:      1
  On failure:       pause
  Monitoring Period: 5s
  Max failure ratio: 0
  Rollback order:   stop-first
ContainerSpec:
  Image:            redis:7@sha256:fbdbaea47b9ae4ecc2082ecdb4e1cea81e32176ff
d9
  Init:             false
Resources:
Endpoint Mode:     vip
```



Изменение успешно применено к сервису (см. UpdateConfig.Delay).

Обновляем сервис Redis на новую версию используемого образа (была 7, станет 7.4) с применением задержки при обновлении реплик (в 10 секунд):

```
$ docker service update --image redis:7.4 redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service update --image redis:7.4 redis
redis
overall progress: 0 out of 2 tasks
1/2: ready
2/2:
█
```

Наблюдаем задержку между обновлениями репликаций сервиса.

Выполним команду для просмотра списка запущенных в Docker-swarm сервисов, чтобы убедиться в успешности обновления версии Redis:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
wo99bund3u3r	nginx	replicated	1/1	nginx:alpine	*:8080->80/tcp
zgir14ih2ztz	redis	replicated	2/2	redis:7.4	



Версия Redis успешно обновлена до V7.4.

Отрабатываем Rollback обновления на последнюю стабильную версию:

```
$ docker service rollback redis
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service rollback redis
redis
rollback: manually requested rollback
overall progress: rolling back update: 2 out of 2 tasks
1/2: running
2/2: running
verify: Service redis converged
```

Откат изменений последнего Rolling-update был произведен.

Выполним команду для просмотра списка запущенных в Docker-swarm сервисов, чтобы убедиться в успешности Rollback обновления Redis назад к версии 7:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
ID                NAME      MODE          REPLICAS  IMAGE          PORTS
wo99bund3u3r     nginx    replicated    1/1        nginx:alpine  *:8080->80/tcp
zgir14ih2ztz     redis    replicated    2/2        redis:7
```



Rollback успешно выполнен.

Дескалируем сервис Redis до 0 репликаций:

```
$ docker service scale redis=0
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service scale redis=0
redis scaled to 0
overall progress: 0 out of 0 tasks
verify: Service redis converged
```

Скалирование успешно завершено.

Выполним команду для просмотра списка запущенных в Docker swarm сервисов, чтобы убедиться в успешности дескалирования сервиса Redis:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
ID                NAME      MODE          REPLICAS  IMAGE          PORTS
wo99bund3u3r     nginx    replicated    1/1        nginx:alpine   *:8080->80/tcp
zg1r14ih2ztz     redis    replicated    0/0        redis:7
```



Сервис успешно дескалирован до 0 репликаций.

Выполним команду просмотра запущенных Docker-контейнеров:

```
$ docker ps
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
76c706d4f2ad   nginx:alpine   "/docker-entrypoint...." About an hour ago Up About an hour
80/tcp        nginx.1.mmqx2qvtg8bprq@ihqsqapbw
```

Docker-контейнеры Redis отсутствуют в выводе команды просмотра запущенных контейнеров.

Выполним команду удаления сервисов nginx и Redis в кластере Docker-swarm:

```
$ docker service rm redis nginx
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service rm redis nginx
redis
nginx
```



Сервисы успешно удалены.

Выполним команду просмотра запущенных в Docker-swarm сервисов, чтобы убедиться в отсутствии запущенных сервисов Nginx и Redis:

```
$ docker service ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
----	------	------	----------	-------	-------

Сервисы отсутствуют в выводе команды просмотра запущенных сервисов.

Выполним команду просмотра запущенных Docker-контейнеров:

```
$ docker ps
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
----	------	------	----------	-------	-------

```
filipp@user-IdeaPad-3-15IAU7:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
filipp@user-IdeaPad-3-15IAU7:~$
```



Убеждаемся в том, что ни одного Docker-контейнера не запущено.

Выводим из кластера узел кластера (при условии, что кластер состоял из одного узла, кластер будет остановлен и удален):

```
$ docker swarm leave -f
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker swarm leave -f  
Node left the swarm.
```

Единственный узел кластера был выведен из эксплуатации.

Выполним команду просмотра статуса узлов запущенного кластера:

```
$ docker node ls
```

```
filipp@user-IdeaPad-3-15IAU7:~$ docker node ls  
Error response from daemon: This node is not a swarm manager. Use "docker swarm init" or "docker swarm join" to connect this node to swarm and try again.
```

Убеждаемся, что кластер Docker-swarm был успешно остановлен и удален.